

好的，下面是对 `PageAgent.ai` 方法调用链路的深入分析，结合了 `demo-new-tab.ts` 的示例和 `DEBUGGING_SUMMARY.md` 的启示，最终形成一份 Markdown 技术文档。

Midscene `PageAgent.ai` 方法调用链路分析

本文档旨在详细解析 `midscene` 项目中 `PageAgent` 类下 `ai` 方法的内部工作流程。分析将从用户通过 `agent.ai("...")` 发起一个自然语言指令开始，一直到系统如何理解该指令、与大语言模型 (LLM) 交互、执行页面操作，并最终生成执行报告。

分析主要基于 `packages/web-integration/src/common/agent.ts` 中的 `PageAgent` 类，并参考了成功执行的 `bridge-demo/demo-new-tab.ts` 示例以及 `midscene_python_bridge/DEBUGGING_SUMMARY.md` 中关于与 Chrome 扩展交互的经验。

1. 调用入口 (`demo-new-tab.ts`)

用户通过 `AgentOverChromeBridge` (或其他 `PageAgent` 的子类或包装类) 实例调用 `ai` 方法。在 `demo-new-tab.ts` 示例中：

```
// bridge-demo/demo-new-tab.ts, L13
await agent.ai('type "AI 101" and hit Enter');
```

这里的 `taskPrompt` 是字符串 `"type \"AI 101\" and hit Enter"`，`type` 参数由于缺省，默认为 `'action'`。

2. `PageAgent.ai(taskPrompt, type)` 方法

该方法是 `PageAgent` 类处理 AI 指令的统一入口。

- 文件路径: `packages/web-integration/src/common/agent.ts`
- 代码定位:

```
async ai(taskPrompt: string, type = 'action') { // L447
  if (type === 'action') { // L448
    return this.aiAction(taskPrompt); // L449
  }
  if (type === 'query') {
    return this.aiQuery(taskPrompt); // L452
  }
  if (type === 'assert') {
    return this.aiAssert(taskPrompt); // L456
  }
  if (type === 'tap') {
    return this.aiTap(taskPrompt); // L460
  }
  throw new Error(
    `Unknown type: ${type}, only support 'action', 'query', 'assert', 'tap'`,
  );
}
```

```
);
}
```

- **逻辑:** 根据传入的 `type` 参数，将任务分发给不同的处理方法。对于 `demo-new-tab.ts` 的调用，由于 `type` 是 `'action'`，它会调用 `this.aiAction(taskPrompt)`。

3. PageAgent.aiAction(taskPrompt) 方法

这是处理通用 AI 行动指令的核心方法。

- **文件路径:** `packages/web-integration/src/common/agent.ts`
- **代码定位:**

```
async aiAction(taskPrompt: string) { // L308
  const isVlmUiTars = vlLocateMode() === 'vlm-ui-tars'; // L310
  const matchedCache = isVlmUiTars // L311
    ? undefined
    : this.taskCache?.matchPlanCache(taskPrompt); // L313

  if (matchedCache && this.taskCache?.isCacheResultUsed) { // L314
    // 如果缓存命中且 taskCache 配置为使用缓存结果
    const yaml = matchedCache.cacheContent?.yamlWorkflow; // L322
    return this.runYaml(yaml); // L323 (执行缓存的 YAML 工作流)
  }

  // 调用 PageTaskExecutor 执行 action
  const { output, executor } = await (isVlmUiTars // L326
    ? this.taskExecutor.actionToGoal(taskPrompt) // L327 (VLM UI TARS 模式)
    : this.taskExecutor.action(taskPrompt,
      this.opts.aiActionContext)); // L328 (标准模式)

  // 更新缓存
  if (this.taskCache && output?.yamlFlow) { // L331
    // ... (缓存更新逻辑, 将新生成的 yamlFlow 存入缓存) ... L332-L345
  }

  this.afterTaskRunning(executor); // L348 (处理执行结果, 生成报告)
  return output; // L349
}
```

- **逻辑:**
 1. **缓存检查 (L310-L323):**
 - 首先判断是否处于 `vlm-ui-tars` 模式 (通过 `vlLocateMode()` 获取，来自 `@midscene/shared/env`)。
 - 如果不是 `vlm-ui-tars` 模式，则尝试从 `this.taskCache` (一个 `TaskCache` 实例，L110) 中匹配 `taskPrompt` 的缓存计划。
 - 如果缓存命中且配置允许使用缓存结果，则直接执行缓存的 YAML 工作流 (`this.runYaml(yaml)` L470)。

2. 核心任务执行 (L326-L328):

- 如果缓存未命中或不使用缓存, 则通过 `this.taskExecutor` (一个 `PageTaskExecutor` 实例, L112) 来执行任务。
- 根据是否为 `vlm-ui-tars` 模式, 调用 `this.taskExecutor.actionToGoal(taskPrompt)` 或 `this.taskExecutor.action(taskPrompt, this.opts.aiActionContext)`。
- `PageTaskExecutor` (定义在 `../common/tasks.ts`) 负责与 LLM 交互生成执行计划, 并实际执行这些计划。

3. 缓存更新 (L331-L345):

- 如果 `taskCache` 启用且 `taskExecutor` 的执行结果 `output` 中包含 `yamlFlow` (由 LLM 生成的执行计划), 则将此计划更新或添加到缓存中。

4. 结果处理与报告 (L348):

- 调用 `this.afterTaskRunning(executor)` 处理执行结果, 包括记录 `dump` 数据和生成报告。

5. 返回 `output` (通常包含 LLM 的思考过程、生成的 YAML 工作流或直接结果)。

4. PageTaskExecutor.action (或 actionToGoal) 内部流程

`PageTaskExecutor` 类 (定义于 `packages/web-integration/src/common/tasks.ts`) 是任务规划与执行的核心。它接收用户指令, 通过与 LLM 交互生成执行计划 (一系列 `PlanningAction`), 然后将这些计划转换为可执行的任务 (`ExecutionTaskApply`), 并最终在浏览器页面上执行这些任务。

下面详细分析其核心方法 `action` 和 `actionToGoal` 的工作流程。

4.1. PageTaskExecutor.action(userPrompt, actionContext) (标准模式)

此方法处理标准模式下的 AI 行动指令。

- 文件路径: `packages/web-integration/src/common/tasks.ts`
- 代码定位: `async action(userPrompt: string, actionContext?: string)` (L809)
- 核心逻辑:
 1. 初始化 **Executor**: 创建一个 `Executor` 实例 (来自 `@midscene/core`) 用于记录和管理整个 `action` 的执行过程 (L815)。
 2. 循环规划与执行 (L822-L871):
 - 该方法采用一个循环 (`while (planningTask)`) 来支持多轮规划。如果 LLM 一次未能生成完整的计划 (由 `planResult.more_actions_needed_by_instruction` 指示), 则会进入下一轮规划。
 - 防止无限循环: 通过 `replanCount` 和 `replanningCountLimit` (L823, L40) 限制最大重规划次数。
 - a. 创建规划任务 (**planningTaskFromPrompt**) (L821, L865):
 - 调用 `this.planningTaskFromPrompt(userPrompt, logList, actionContext)` (L624) 来创建一个 `ExecutionTaskPlanningApply` 类型的任务。
 - **planningTaskFromPrompt** 内部 (L624-L730):
 - 准备规划上下文 (**setupPlanningContext**) (L634):
 - 调用 `this.insight.contextRetrieverFn('locate')` (L572) 获取当前页面的 `WebUIContext` (包含截图、DOM树等)。

- 这个 `pageContext` 会被记录到 `ExecutionTaskPlanning` 中。

- 这是与 LLM 直接交互进行任务规划的地方。它调用了从 `@midscene/core` 导入的 `plan` 函数 (详见第6节关于 `@midscene/core` 的分析)。

- **LLM 输出 (planResult):** 一个 PlanningAIResponse 对象 (来自 @midscene/core), 包含:

- **actions: PlanningAction[]**: LLM 生成的行动步骤列表。
- **log: string**: LLM 的思考过程和决策日志。
- **more_actions_needed_by_instruction: boolean**: 指示当前计划是否完整，或是否需要基于 **log** 进行下一轮规划。
- **yamlFlow: MidsceneYamlFlowItem[]**: LLM 生成的等效 YAML 工作流。
- **error: string**: LLM 规划时发生的错误。
- **usage: AIUsageInfo**: LLM token 使用信息。
- **rawResponse**: LLM 的原始响应。
- **sleep: number**: LLM 建议的等待时间。

- 从 `planResult` 中提取 `actions`。

- **动作预处理 (finalActions) (L660-L693):** 清理和规范化 LLM 返回的 actions。例如，确保 Tap, Hover, Input 等交互动作有关联的 locate 信息，如果缺少则可能标记为规划错误。对于连续的 Locate，会处理 bbox 的收集，避免重复定位。

- 如果 LLM 建议了 **sleep**，则添加一个 **Sleep** 动作。
- 如果 **actions** 为空且不是因为需要 **sleep**，则抛出错误。

- 返回包含 actions、yamlFlow、log 等信息的对象。

- 将创建的 `planningTask` 添加到 `taskExecutor` 并执行 (`await taskExecutor.flush()`)。

- 执行结果 (planResult) 即为 planningTaskFromPrompt 中 LLM 返回的 PlanningAIResponse。

- 如果执行出错，则流程终止。

- 从 `planResult` 中提取 `yamlFlow` 并累加到 `action` 方法的 `yamlFlow` 变量中，该变量最终会返回给 `PageAgent.aiAction` 用于缓存。

- 调用 `this.convertPlanToExecutable(plans)` (L149) 将 LLM 返回的 `PlanningAction[]` (即 `planResult.actions`) 转换为 `ExecutionTaskApply[]`。这一步在下面的 **5. `convertPlanToExecutable` 详解** 详细描述。

- 将转换后的可执行任务列表追加到 `taskExecutor`。
 - e. 执行转换后的任务 (L847):
 - `await taskExecutor.flush()` 执行这些具体的页面操作任务。
 - 如果出错, 流程终止。
 - f. 记录日志并判断是否继续循环 (L853-L869):
 - 将 `planResult.log` 添加到 `logList`。
 - 如果 `planResult.more_actions_needed_by_instruction` 为 `false`, 表示规划完成, 跳出循环。
 - 否则, 准备下一轮规划, 将 `logList` 作为输入, `replanCount` 增加。
3. 返回结果 (L873-L878):
- 返回包含累积的 `yamlFlow` (用于缓存) 和 `taskExecutor` (包含所有执行记录) 的对象。

4.2. `PageTaskExecutor.actionToGoal(userPrompt)` (VLM UI TARS 模式)

此方法用于视觉语言模型 (VLM) 模式, 如 `vlm-ui-tars`。

- 文件路径: `packages/web-integration/src/common/tasks.ts`
- 代码定位: `async actionToGoal(userPrompt: string)` (L886)
- 核心逻辑: 与 `action` 方法类似, 它也采用循环规划和执行的模式, 但其规划任务的创建和 LLM 调用方式不同。
 1. 初始化 `Executor` 和对话历史 (L892, L893):
 - `this.conversationHistory = []` 用于存储与 VLM 的多轮对话。
 2. 循环规划与执行 (L897-L938):
 - a. 创建规划任务 (`planningTaskToGoal`) (L899):
 - 调用 `this.planningTaskToGoal(userPrompt)` (L733) 创建规划任务。
 - `planningTaskToGoal` 内部 (L733-L790):
 - 准备规划上下文 (`setupPlanningContext`) (L740): 同标准模式, 获取 `pageContext`。
 - 处理图像输入 (L742-L750):
 - 调用 `resizeImageForUiTars` (来自 `@midscene/core/ai-model`) 处理页面截图, 使其符合 VLM 的输入要求。
 - 将处理后的图像和用户指令作为用户消息添加到 `this.conversationHistory` (通过 `this.appendConversationHistory` (L1092) 管理对话历史长度)。
 - 调用 VLM 规划函数 (`vlmPlanning`) (L753):
 - 这是与 VLM 直接交互进行任务规划的地方。它调用了从 `@midscene/core/ai-model` 导入的 `vlmPlanning` 函数 (详见第6节关于 `@midscene/core` 的分析)。
 - LLM (VLM) 输入: 用户指令、包含当前截图的对话历史、页面尺寸。
 - LLM (VLM) 输出 (`planResult`):
 - `actions: PlanningAction[]`: VLM 生成的行动步骤。
 - `action_summary: string`: VLM 对其所执行动作的文字总结, 会作为助手消息添加到 `this.conversationHistory` (L760)。
 - `yamlFlow: MidsceneYamlFlowItem[]`: 等效的 YAML 工作流。
 - 返回包含 `actions`、`yamlFlow` 等信息的对象。

- b. 执行规划任务 (L901): 同标准模式。
 - c. 收集 YAML Flow (L907): 同标准模式。
 - d. 将规划动作转换为可执行任务 (**convertPlanToExecutable**) (L912): 同标准模式。
 - e. 执行转换后的任务 (L919): 同标准模式。
 - f. 判断是否结束 (L927-L930):
 - 如果 VLM 返回的 `plans[0].type` 是 'Finished', 则表示任务完成, 跳出循环。
- VLM 模式通常期望模型自行判断任务是否完成。

3. 返回结果 (L940-L945): 同标准模式。

4.3. 其他相关辅助方法 (**PageTaskExecutor**)

- **query, boolean, number, string** (L1001-L1032):
 - 这些方法用于执行从页面提取特定类型信息的任务。
 - 它们内部调用 `this.createTypeQueryTask` (L971)。
 - `createTypeQueryTask` 的 `executor` 调用 `this.insight.extract(demandInput)` (L990), 由 `@midscene/core` 的 `Insight` 模块处理提取逻辑 (可能涉及 LLM)。
- **assert** (L1034-L1055):
 - 用于执行断言任务。它将断言请求转换为一个 `Assert` 类型的 `PlanningAction`, 然后通过 `convertPlanToExecutable` 和 `taskExecutor.flush()` 执行。
- **waitFor** (L1116-L1166):
 - 用于等待某个条件满足。它在一个循环中反复执行一个 '`AssertWithoutThrow`' 类型的规划动作, 直到断言通过或超时。

5. **convertPlanToExecutable(plans: PlanningAction[])** 详解

此方法是将 LLM (或 VLM) 生成的抽象 `PlanningAction` 列表转换为具体的、可被 `Executor` 执行的 `ExecutionTaskApply` 对象列表的关键。

- 文件路径: `packages/web-integration/src/common/tasks.ts`
- 代码定位: `private async convertPlanToExecutable(plans: PlanningAction[])` (L149)
- 逻辑:
 1. 遍历 `plans` 数组中的每一个 `PlanningAction`。
 2. 根据 `plan.type` 将其映射为一个具体的 `ExecutionTaskApply` 对象, 并定义其 `executor` 函数, 该函数包含了实际的页面操作逻辑。
 - **plan.type === 'Locate'** (L153-L254):
 - 创建一个 `ExecutionTaskInsightLocateApply` 任务。
 - 其 `executor` 负责定位元素:
 - 获取页面上上下文 (L178): `await this.insight.contextRetrieverFn('locate')`。
 - 缓存检查 (L187-L213):
 - 尝试从 `this.taskCache` 使用 `param.prompt` (定位提示) 匹配缓存的 XPath (`locateCacheRecord?.cacheContent?.xpath`)。
 - 如果命中缓存且允许使用缓存 (`this.taskCache?.isCacheResultUsed`), 则尝试通过 `this.page.evaluateJavaScript` 执行 `midscene_element_inspector.getElementInfoByXPath` 来获取元素。 `getElementInfosScriptContent` (来自

@midscene/shared/fs) 提供了 midscene_element_inspector 的脚本内容。

- **无缓存或缓存未命中 (L221-L228):**
 - 首先尝试 `matchElementFromPlan(param, pageContext.tree)` (L223), 该函数 (定义在 `packages/web-integration/src/common/utils.ts`) 尝试直接从当前 `pageContext.tree` (DOM树) 中根据 `plan.locate` 的信息 (如 ID, bbox) 匹配元素。
 - 如果仍未找到, 则调用 `this.insight.locate(param, { context: pageContext })` (L227), 这会触发 @midscene/core 中的 `Insight` 模块进行更复杂的 AI 定位 (可能再次调用 LLM 或使用视觉模型)。
- **缓存更新 (L233-L245):** 如果成功定位到元素且未使用缓存, 会尝试获取该元素的 XPath (通过 `this.getElementXPath` (L94), 内部也使用 `evaluateJavaScript` 和 `midscene_element_inspector.getXpathsById`), 并更新 `this.taskCache`。
- 如果最终未找到元素, 则抛出错误。
- **`plan.type === 'Assert' 或 'AssertWithoutThrow'` (L255-L292):**
 - 创建一个 `ExecutionTaskApply` 任务。
 - 其 `executor` 调用 `this.insight.assert(assertPlan.param.assertion)` (L270) 来进行断言。`this.insight.assert` 可能会再次调用 LLM 来判断断言是否成立。
 - 如果断言失败且类型是 `'Assert'`, 则抛出错误。
- **`plan.type === 'Input'` (L293-L310):**
 - 创建一个 `ExecutionTaskActionApply` 任务。
 - 其 `executor` 首先会尝试 `this.page.clearInput(element)` (如果 `element` 已定位), 然后调用 `this.page.keyboard.type(taskParam.value)` 来输入文本。这里的 `this.page` 是 `WebPage` 接口的实例 (如 `PuppeteerWebPage`, `ChromeExtensionProxyPage` 等), 其实际实现会调用相应的浏览器控制方法。
- **`plan.type === 'KeyboardPress'` (L311-L325):**
 - 其 `executor` 调用 `getKeyCommands(taskParam.value)` (L319, 来自 `./ui-utils`) 将按键名称 (如 "Enter") 转换为具体指令, 然后调用 `this.page.keyboard.press(keys)`。
- **`plan.type === 'Tap'` (L326-L336):**
 - 其 `executor` 调用 `this.page.mouse.click(element.center[0], element.center[1])`, 在定位到的元素的中心点执行点击。
- **`plan.type === 'Drag'` (L337-L351):**
 - 其 `executor` 调用 `this.page.mouse.drag(taskParam.start_box, taskParam.end_box)`。
- **`plan.type === 'Hover'` (L352-L362):**
 - 其 `executor` 调用 `this.page.mouse.move(element.center[0], element.center[1])`。
- **`plan.type === 'Scroll'` (L363-L420):**
 - 其 `executor` 根据 `taskParam.scrollType` 和 `taskParam.direction` 调用 `this.page` 对象的各种滚动方法 (如 `scrollUntilTop`, `scrollDown` 等)。

- `plan.type === 'Sleep' (L421-L431):`
 - 其 `executor` 调用 `await sleep(taskParam?.timeMs || 3000)`。
 - `plan.type === 'Error' (L432-L444):`
 - 其 `executor` 直接抛出错误。
 - `plan.type === 'Finished', 'ExpectedFalsyCondition', 'AndroidHomeButton', 'AndroidBackButton', 'AndroidRecentAppsButton' (L445-L513):`
 - 这些分别对应不同的结束状态或特定的设备操作，其 `executor` 会执行相应的逻辑，例如调用 `this.page.home()` (L477) 或 `this.page.back()` (L491) (这些方法在 `AndroidDevicePage` 中定义)。
3. 截图包装 (`prependExecutorWithScreenshot`) (L518-L525):
- 对于所有 'Action' 类型的任务，会使用 `this.prependExecutorWithScreenshot` (L128) 进行包装。
 - `prependExecutorWithScreenshot` 内部 (L128-L147):
 - 在原 `executor` 执行之前和之后 (如果 `appendAfterExecution` 为 `true`，通常对最后一个 Action 任务) 调用 `this.recordScreenshot()` (L78) 来捕获页面截图。
 - `this.recordScreenshot()` (L78) 调用 `this.page.screenshotBase64()` 获取截图，并将其存入任务的 `recorder` 数组。
 - 对于 Action 任务，在执行后还会尝试等待网络空闲 (`((this.page as PuppeteerWebPage).waitUntilNetworkIdle())`) 并固定等待一小段时间，以确保页面加载完成。

6. 页面操作 (`this.page.*`)

在 `convertPlanToExecutable` 中，所有具体的页面交互最终都委托给了 `this.page` 对象的方法。`this.page` 是 `WebPage` 接口 (定义在 `packages/web-integration/src/common/page.ts`) 的一个实例。

- 对于 **Puppeteer/Playwright** 环境，`this.page` 可能是 `PuppeteerWebPage` 或类似的实现，其方法会直接调用 Puppeteer/Playwright 的 API 来控制浏览器。
- 对于 **Chrome 扩展 Bridge 模式** (如 `demo-new-tab.ts` 的场景)，`this.page` 是 `ExtensionBridgePageBrowserSide` 的一个实例 (通过 `AgentOverChromeBridge` 间接使用) 或 `ChromeExtensionProxyPage` (在 `Popup` 中直接使用)。
 - `ChromeExtensionProxyPage` (位于 `packages/web-integration/src/chrome-extension/page.ts`) 的方法通常会通过 `chrome.debugger.sendCommand` 发送 CDP (Chrome DevTools Protocol) 命令给附加的标签页，或通过 `chrome.scripting.executeScript` 在目标页面执行 JavaScript 代码来实现交互。这与 `DEBUGGING_SUMMARY.md` 中描述的 Python Bridge 与扩展的交互方式类似。
 - 例如，`page.mouse.click(x, y)` 可能最终转换为 CDP 的 `Input.dispatchMouseEvent` 命令。
 - `page.keyboard.type(text)` 可能通过 CDP 的 `Input.dispatchKeyEvent` 或 `Runtime.evaluate` 来模拟输入。
 - `page.screenshotBase64()` 可能使用 CDP 的 `Page.captureScreenshot`。

7. 用户意图理解与 Prompt 生成 (聚焦 `@midscene/core`)

@midscene/core 包是 Midscene 理解用户自然语言指令并将其转化为可执行计划的核心。这一过程主要涉及到对用户意图的解析、页面上下文的理解、以及据此生成有效的大语言模型 (LLM) 或视觉语言模型 (VLM) prompt。

7.1. 核心流程概述

7.1.1. 理解用户意图的核心入口 (plan 和 vlmPlanning)

用户意图的理解和初步处理主要发生在 PageTaskExecutor 调用 @midscene/core 中的规划函数时：

- **标准 LLM 模式:** 通过 plan 函数 (位于 packages/core/src/ai-model/llm-planning.ts)。
- **VLM (如 UI-TARS) 模式:** 通过 vlmPlanning 函数 (位于 packages/core/src/ai-model/ui-tars-planning.ts)。

这两个函数是用户原始指令 (userInstruction) 进入 @midscene/core AI 模块的起点。

7.1.2. 通用步骤：页面上下文处理 (describeUserPage, descriptionOfTree)

在进行规划前，需要将当前页面的状态（截图、DOM结构、元素信息）转化为LLM/VLM能够理解的格式。

- **describeUserPage (位于 packages/core/src/ai-model/prompt/util.ts):**
 - 接收 UIContext (包含截图、DOM树 ElementTreeNode[], 页面尺寸 Size)。
 - 提取页面尺寸。
 - 将 ElementTreeNode[] 扁平化为元素列表，并创建ID到元素的映射 (idElementMap)，方便后续通过ID查找。
 - 调用 descriptionOfTree (位于 packages/core/src/tree.ts) 将 ElementTreeNode[] 转换为类似HTML的文本字符串 (contentTree)。此字符串包含元素的ID、markerId (若有，用于截图标记)、位置、尺寸、属性和文本内容。这为纯文本LLM提供了丰富的结构化信息。
 - 返回一个 pageDescription 字符串（包含尺寸和 contentTree）和辅助函数（如 elementById）。

7.1.3. 通用步骤：图像处理

- **标准 LLM 模式 (非 vlMode):**
 - 调用 markupImageForLLM (位于 packages/core/src/ai-model/common.ts)。
 - 该函数会使用 compositeElementInfoImg (来自 @midscene/shared/img) 在原始截图上根据非文本元素的位置绘制标记框，生成一个新的Base64图像。这有助于LLM将文本描述的元素与图像中的视觉标记关联起来。
- **VLM 模式:**
 - **Qwen-VL:** paddingToMatchBlockByBase64 (来自 @midscene/shared/img) 用于对截图进行填充，以符合Qwen-VL模型的输入要求。
 - **UI-TARS (v1.5):** resizeImageForUiTars (位于 packages/core/src/ai-model/ui-tars-planning.ts) 会检查图像像素是否超限，如果超限则进行缩放。
- **通用:** warnGPT4oSizeLimit (位于 packages/core/src/ai-model/common.ts) 会检查图像尺寸，若对特定模型（如GPT-4o）过大则发出警告。

7.1.4. 通用步骤：调用 LLM/VLM (callAiFn)

- `callAiFn` (位于 `packages/core/src/ai-model/common.ts`) 是调用AI模型的统一入口，它内部调用 `callToGetJSONObject` (位于 `packages/core/src/ai-model/service-caller/index.ts`)。
- `callToGetJSONObject` 内部：
 - `createChatClient`: 初始化OpenAI或Anthropic的客户端，处理API Key、代理、Azure配置等。
 - `call`: 实际执行对LLM/VLM API的请求。传入模型名称、消息列表 (包含系统指令、用户指令和处理后的图像)、温度、最大token数等参数。
 - 对于需要JSON输出的任务 (如规划、定位)，会设置 `response_format`，并可能提供JSON Schema (如 `planSchema`, `locatorSchema`) 来规范输出。
 - `safeParseJson`: 解析LLM/VLM返回的可能是非严格JSON的字符串。

7.1.5. 通用步骤：响应处理与 YAML 生成

- 从LLM/VLM的响应中提取结构化数据 (如 `actions` 列表)。
- 调用 `buildYamlFlowFromPlans` (位于 `packages/core/src/ai-model/common.ts`) 将 `PlanningAction[]` 转换为等效的YAML工作流字符串，便于缓存和复现。

7.2. 标准 LLM 模式下的规划 (`plan` 函数详解)

位于 `packages/core/src/ai-model/llm-planning.ts`。

7.2.1. 系统 Prompt 构建 (`systemPromptToTaskPlanning` - 非 `vLMMode`)

- 调用 `systemPromptToTaskPlanning` (来自 `packages/core/src/ai-model/prompt/llm-planning.ts`)。
- 对于非 `vLMMode`：
 - 使用 `systemTemplateOfLLM` 和 `outputTemplate`。
 - `systemTemplateOfLLM`: 定义LLM的角色 (UI自动化专家)、目标 (分解指令、定位元素)、工作流程、约束 (如必须基于页面上下文、不重复历史操作、JSON输出)、支持的动作类型 (Tap, Hover, Input, KeyboardPress, Scroll等) 及其参数格式 (如 `locate: { id: string, prompt: string }`)。
 - `outputTemplate`: 提供输出JSON的格式示例，包括 `actions` 数组、`log` (LLM思考过程)、`more_actions_needed_by_instruction` 和 `error` 字段。
 - `samplePageDescription` (来自 `packages/core/src/ai-model/prompt/util.ts`) 用于在Prompt中展示页面描述的格式。

7.2.2. 用户 Prompt 构建 (`automationUserPrompt` - 非 `vLMMode`)

- 调用 `automationUserPrompt` (来自 `packages/core/src/ai-model/prompt/llm-planning.ts`)。
- 它将 `pageDescription` (7.1.2中生成) 和 `taskBackgroundContext` 组合。
- `generateTaskBackgroundContext` 格式化用户原始指令、可选的上一轮日志 (`opts.log`) 和开发者提供的额外上下文 (`opts.actionContext`)。
- 对于非 `vLMMode`，用户Prompt模板为：

```
pageDescription:
=====
```

```
{pageDescription}
=====

{taskBackgroundContext}
```

7.2.3. LLM 响应处理 (非 vLMMode)

- ID 校正: 如果 `action.locate.id` 存在, 会通过 `elementById` (来自 `describeUserPage` 的辅助函数) 查找元素, 确保ID有效性, 防止模型返回幻想的 `indexId`。
- 如果 `actions` 为空但 `more_actions_needed_by_instruction` 为 `true` 且未建议 `sleep`, 则发出警告。

7.3. VLM 模式下的规划 (vLMPlanning 函数详解)

位于 `packages/core/src/ai-model/ui-tars-planning.ts`。

7.3.1. 系统 Prompt 构建 (getUiTarsPlanningPrompt)

- 调用 `getUiTarsPlanningPrompt` (来自 `packages/core/src/ai-model/prompt/ui-tars-planning.ts`)。
- 此Prompt定义VLM的角色 (GUI Agent)、输出格式 ("Thought: ... Action: ...")、支持的动作空间 (click, drag, type, hotkey, scroll, wait, finished, call_user), 并强调坐标格式 (如 `start_box='[x1, y1, x2, y2]'`) 和语言要求 (根据 `getTimeZoneInfo` 判断中英文)。
- 用户原始指令 `userInstruction` 会追加到此系统Prompt后。

7.3.2. VLM 响应解析与转换 (actionParser)

- `convertBboxToCoordinates`: 将VLM可能返回的 `<bbox>x1 y1 x2 y2</bbox>` 形式的bbox转换为坐标 (x,y)。
- `actionParser` (来自 `@ui-tars/action-parser`): 关键解析库, 接收VLM预测文本, 根据动作语法解析为结构化动作对象列表 (`parsed`)。
- 解析后的动作会进一步转换为Midscene内部的 `PlanningAction[]` 格式。例如, VLM的 `click` 会转换为一个 `Locate` 和一个 `Tap`。hotkey 使用 `transformHotkeyInput`。

7.4. 特定任务的 Prompt 构建

除了主规划流程, `@midscene/core` 在执行具体子任务 (定位、断言、提取) 时也会与LLM/VLM交互, 并使用特定的Prompt。

7.4.1. 定位 (AiLocateElement 和 AiLocateSection)

- **AiLocateElement** (位于 `packages/core/src/ai-model/inspect.ts`):
 - 系统 Prompt (`systemPromptToLocateElement`):
 - 来自 `packages/core/src/ai-model/prompt/llm-locator.ts`。
 - `vLMMode`: 指示模型识别截图元素并返回其 `bbox`。强调bbox格式。
 - 非 `vLMMode`: 指示模型分析截图和文本描述, 返回匹配元素的 `id`。强调使用真实 `id` 而非 `indexId`。提供JSON输出格式和示例。
 - 用户 Prompt (`findElementPrompt`):

- 模板: "Here is the item user want to find:\n=====\n{targetElementDescription}\n=====\n\n{pageDescription}"
- targetElementDescription: 用户对目标的描述。
- pageDescription: 来自 describeUserPage。
- 可选区域搜索 (AiLocateSection):
 - 如果先调用 AiLocateSection (用于粗略定位区域), 其结果 (options.searchConfig) 会用于 AiLocateElement, 使其在区域截图上进行精细定位。
 - AiLocateSection 的系统 Prompt (systemPromptToLocateSection) 指示模型找到包含目标和参考元素的区域 bbox。

7.4.2. 断言 (AiAssert)

- AiAssert (位于 packages/core/src/ai-model/inspect.ts):
 - 系统 Prompt (systemPromptToAssert):
 - 来自 packages/core/src/ai-model/prompt/assertion.ts。
 - 指示模型扮演测试工程师, 根据截图判断用户断言真伪。
 - 根据 isUITars 选择不同JSON输出格式 (defaultAssertionResponseJsonFormat 或 uiTarsAssertionResponseJsonFormat), 主要区别在于 thought 字段的语言和格式。
 - 用户 Prompt:
 - 包含截图和用户断言文本: "Here is the assertion. Please tell whether it is truthy according to the screenshot.\n=====\n{assertion}\n=====\n"

7.4.3. 数据提取 (AiExtractElementInfo)

- AiExtractElementInfo (位于 packages/core/src/ai-model/inspect.ts):
 - 系统 Prompt (systemPromptToExtract):
 - 来自 packages/core/src/ai-model/prompt/extraction.ts。
 - 指示模型扮演UI设计和测试专家, 根据截图、页面内容和 DATA_DEMAND 提取数据。要求返回JSON格式: { data: any, errors: [] }。
 - 用户 Prompt (extractDataPrompt):
 - 模板: "pageDescription: {pageDescription}\n\nExtract the following data and place it in the \\data\\ field...\nDATA_DEMAND start:\n=====\n{dataKeys}\n\n{dataQuery}\n=====\nDATA_DEMAND ends.\n"
 - pageDescription: 页面结构文本。
 - dataKeys: 对期望返回数据键的提示。
 - dataQuery: 用户具体的提取要求。

7.5. Prompt 工程总结

@midscene/core 通过精心设计的 Prompt 工程来指导 LLM/VLM 理解用户意图并执行任务：

1. **结构化页面信息**: 将 DOM 树、元素属性、截图等信息处理成 LLM/VLM 更易理解的格式 (文本描述、标记图像、区域截图)。
2. **角色扮演与任务定义**: 系统 Prompt 明确告知模型其角色、目标、可用动作、输出格式和约束。
3. **上下文注入**: 用户指令、历史日志、开发者提供的附加上下文被整合到用户 Prompt 中，为模型提供决策所需的所有相关信息。
4. **模式适配**: 针对纯文本 LLM 和多模态 VLM，采用不同的 Prompt 策略和图像处理方法。VLM 更依赖直接的视觉信息和简洁指令，而 LLM 则需要更详细的文本描述和元素标记。
5. **迭代与细化**: 对于复杂任务，支持多轮规划，上一轮的日志 (log) 会作为下一轮的输入，帮助模型逐步逼近目标。对于定位任务，可以使用 `AiLocateSection` 进行初步的粗略定位，再进行精细定位。
6. **严格的输出格式**: 通过在 Prompt 中指定 JSON Schema 或详细的 JSON 格式示例，引导模型产生结构化、可解析的输出，便于后续程序处理。

通过这些机制，@midscene/core 能够有效地将模糊的自然语言指令转化为精确的、可执行的自动化步骤。

7.6. 示例：处理"点击第一个按钮"指令 (标准 LLM 模式)

本节将以用户指令"点击第一个按钮"为例，详细解析在标准LLM模式（非vLM模式）下，@midscene/core内部如何构建Prompt并预期LLM的响应。

7.6.1. 任务与假设上下文

- 用户指令: "点击第一个按钮"
- 假设的 `UIContext`:
 - `screenshotBase64`: (一个包含至少两个按钮的页面截图的Base64编码，此处省略)
 - `size`: { `width`: 800, `height`: 600 }
 - `tree` (简化的DOM结构):

```
{
  "node": null, // Root node
  "children": [
    {
      "node": {
        "id": "btn_1_abc", "indexId": 0,
        "attributes": { "nodeType": "BUTTON Node", "text":
"Button 1" }, "content": "Button 1",
        "rect": { "left": 50, "top": 100, "width": 100, "height":
30 }, "center": [100, 115]
      }, "children": []
    },
    {
      "node": {
        "id": "btn_2_xyz", "indexId": 1,
        "attributes": { "nodeType": "BUTTON Node", "text":
"Second Button" }, "content": "Second Button",
        "rect": { "left": 50, "top": 150, "width": 150, "height":
30 }, "center": [125, 165]
      }, "children": []
    }
  ]
}
```

```
]
}
```

- 假设:
 - 当前为首次规划, `opts.log` (历史日志) 为空。
 - `opts.actionContext` (开发者附加上下文) 为空。
 - 非 `vLMMode`。

7.6.2. 步骤1: `PageTaskExecutor` 调用 `plan`

如前文所述, `PageTaskExecutor.action("点击第一个按钮")` 会在其内部调用 `this.planningTaskFromPrompt`, 后者最终会调用 `@midscene/core` 的 `plan` 函数: `plan("点击第一个按钮", { context: hypotheticalUIContext, pageType: 'web', log: undefined, actionContext: undefined })`

7.6.3. 步骤2: `describeUserPage` 生成 `pageDescription`

`plan` 函数内部首先调用 `describeUserPage(hypotheticalUIContext)`。基于上述假设的 `tree` 和 `size`:

- `descriptionOfTree` 会生成类似如下的 `contentTree`:

```
<button id="btn_1_abc" markerId="0" left="50" top="100" width="100"
height="30">
  Button 1
</button>
<button id="btn_2_xyz" markerId="1" left="50" top="150" width="150"
height="30">
  Second Button
</button>
```

- `describeUserPage` 返回的 `pageDescription` 将是:

```
The size of the page: 800 x 600
Some of the elements are marked with a rectangle in the screenshot,
some are not.
The page elements tree:
<button id="btn_1_abc" markerId="0" left="50" top="100" width="100"
height="30">
  Button 1
</button>
<button id="btn_2_xyz" markerId="1" left="50" top="150" width="150"
height="30">
  Second Button
</button>
```

7.6.4. 步骤3: `systemPromptToTaskPlanning` 生成系统 Prompt

由于是非 `vlMode`, `systemPromptToTaskPlanning` (来自 `packages/core/src/ai-model/prompt/llm-planning.ts`) 会使用 `systemTemplateOfLLM` 和 `outputTemplate`。核心内容如下 (为简洁省略部分示例和细节) :

```
## Role
```

```
You are a versatile professional in software UI automation. Your outstanding contributions will impact the user experience of billions of users.
```

```
## Objective
```

- Decompose the instruction user asked into a series of actions
- Locate the target element if possible
- If the instruction cannot be accomplished, give a further plan.

```
## Workflow
```

```
...
```

```
## Constraints
```

```
...
```

- Respond only with valid JSON. Do not write an introduction or summary or markdown prefix like ````json````.
- If the screenshot and the instruction are totally irrelevant, set reason in the ``error`` field.

```
## About the `actions` field
```

```
The `locate` param is commonly used in the `param` field of the action, means to locate the target element to perform the action, it conforms to the following scheme:
```

```
type LocateParam = {
```

```
  "id": string, // the id of the element found. It should either be the id marked with a rectangle in the screenshot or the id described in the description.
```

```
  "prompt"?: string // the description of the element to find. It can only be omitted when locate is null.
```

```
} | null // If it's not on the page, the LocateParam should be null
```

```
## Supported actions
```

```
Each action has a `type` and corresponding `param`. To be detailed:
```

```
- type: 'Tap'
```

```
  * { locate: LocateParam }
```

```
... (其他动作类型) ...
```

```
## Output JSON Format:
```

```
The JSON format is as follows:
```

```
{
```

```
  "actions": [
```

```
    // ... some actions
```

```
  ],
```

```
  "log": string, // Log what the next actions you can do according to the
```



```
screenshot and the instruction. ...
"more_actions_needed_by_instruction": boolean, // Consider if there is
still more action(s) to do after the action in "Log" is done...
"error"?: string, // Error messages about unexpected situations...
}

## Examples
... (示例部分) ...
```

(实际Prompt会用 `samplePageDescription` 填充示例中的页面描述部分)

7.6.5. 步骤4: `automationUserPrompt` 生成用户 Prompt

- `generateTaskBackgroundContext` 会将用户指令格式化为:

```
Here is the user's instruction:
<instruction>
  <high_priority_knowledge>
    undefined
  </high_priority_knowledge>

  点击第一个按钮
</instruction>
```

(因为 `opts.log` 和 `opts.actionContext` 假设为空)

- `automationUserPrompt` (非 `vlMode`) 会将此 `taskBackgroundContext` 与前面生成的 `pageDescription` 组合, 形成最终的用户 Prompt:

```
pageDescription:
=====
The size of the page: 800 x 600
Some of the elements are marked with a rectangle in the screenshot,
some are not.
The page elements tree:
<button id="btn_1_abc" markerId="0" left="50" top="100" width="100"
height="30">
  Button 1
</button>
<button id="btn_2_xyz" markerId="1" left="50" top="150" width="150"
height="30">
  Second Button
</button>
=====

Here is the user's instruction:
<instruction>
  <high_priority_knowledge>
    undefined
  </high_priority_knowledge>
```

```
    点击第一个按钮
</instruction>
```

7.6.6. 步骤5: 图像处理 (markupImageForLLM)

- `markupImageForLLM` 会被调用, 传入原始截图 (`hypotheticalUIContext.screenshotBase64`) 和DOM树 (`hypotheticalUIContext.tree`)。
- 它会识别出非文本节点 (在此例中是两个按钮), 并在截图的相应位置绘制标记框 (例如, 基于 `markerId` 或元素位置)。处理后的带有标记的图像 (Base64编码) 将作为 `imagePayload`。

7.6.7. 步骤6: LLM 调用

- 系统 Prompt (7.6.4)、用户 Prompt (7.6.5) 和标记后的图像 (7.6.6) 会被封装成一个消息列表, 通过 `callAiFn -> callToGetJSONObject -> call` 发送给 LLM。
- `response_format` 会被设置为要求JSON输出, 并可能使用 `planSchema` (来自 `packages/core/src/ai-model/prompt/llm-planning.ts`) 来指导LLM返回特定结构的JSON。

7.6.8. 步骤7: LLM 预期输出

基于上述输入, LLM被期望分析页面结构和用户指令"点击第一个按钮"。一个理想的、简化的 `PlanningAIResponse` JSON输出可能如下:

```
{
  "actions": [
    {
      "type": "Tap",
      "param": null,
      "locate": {
        "id": "btn_1_abc",
        "prompt": "第一个按钮"
      },
      "thought": "用户要求点击第一个按钮, 根据页面描述, ID为 'btn_1_abc' 的元素是第一个按钮, 其内容为 'Button 1'。"
    }
  ],
  "log": "识别到用户指令为点击第一个按钮, 计划执行Tap操作, 目标元素ID为 'btn_1_abc'。",
  "more_actions_needed_by_instruction": false,
  "error": null,
  "usage": { // 示例 usage 信息
    "prompt_tokens": 500,
    "completion_tokens": 80,
    "total_tokens": 580
  },
  "rawResponse": "...", // LLM的原始字符串响应
  "yamlFlow": [
    {
      "aiTap": "第一个按钮"
    }
  ]
}
```

```

    }
  ]
}

```

说明:

- `actions[0].locate.id`: LLM应从 `pageDescription` 中识别出 "第一个按钮" 对应的是 `btn_1_abc`。
- `actions[0].locate.prompt`: LLM生成的对该元素的描述, 可能与用户指令中的描述相似。
- `log`: LLM的思考过程。
- `more_actions_needed_by_instruction`: 因为指令简单, 一步完成, 所以为 `false`。
- `yamlFlow`: 根据 `actions` 生成的等效YAML。

这个详细示例展示了从用户简单指令到复杂Prompt构建, 再到LLM结构化输出的完整流程。

8. 结果处理与报告生成 (`PageAgent.afterTaskRunning`)

当 `PageTaskExecutor` 的 `action` 或其他方法执行完毕后, 控制权回到 `PageAgent` (例如, 在 `PageAgent.aiAction` 中的 L348)。

- 文件路径: `packages/web-integration/src/common/agent.ts`
- 代码定位: `private afterTaskRunning(executor: Executor, doNotThrowError = false)` (L180)
- 逻辑:
 1. 追加执行记录 (L181):
 - `executor.dump()`: `PageTaskExecutor` 返回的 `Executor` 实例包含了其执行的所有 `ExecutionTask` 的详细记录 (参数、日志、截图、耗时、LLM使用情况等), `dump()` 方法将这些记录序列化为一个 `ExecutionDump` 对象。
 - `this.appendExecutionDump(executor.dump())` (L155): 将这个 `ExecutionDump` 添加到 `this.dump.executions` 数组中。`this.dump` 是一个 `GrouperActionDump` 对象, 用于聚合多次 `ai` 调用或一个复杂 `ai` 调用分解的多个执行单元的结果。
 2. 写出 `Dump` 数据和报告 (L182):
 - `this.writeOutActionDumps()` (L168) 负责将当前的 `GrouperActionDump` 数据持久化。
 - `this.dumpDataString()` (L161): 将 `this.dump` 对象 (包含所有 `ExecutionDump`) 通过 `@midscene/core/utils` 的 `stringifyDumpData` 序列化为 JSON 字符串。
 - `writeLogFile` (L170, 来自 `@midscene/core/utils`): 将此 JSON 字符串写入到文件 (例如, `midscene_run/dump/...json`)。
 - 如果 `generateReport` 选项为 `true` (默认行为), `writeLogFile` 还会 (或其内部逻辑会) 使用 `reportHTMLContent` (L165, 来自 `@midscene/core/utils`) 将 `this.dumpDataString()` 的内容转换为用户友好的 HTML 报告, 并写入文件。最终的 HTML 文件路径如 `demo-new-tab.ts` 控制台日志所示。
 3. 错误处理 (L184-L186): 如果 `executor` 在执行过程中记录了错误状态, 则抛出错误。

9. 整体总结

PageAgent.ai 方法的调用链路是一个复杂但设计精良的过程，它将用户的自然语言指令转化为具体的浏览器操作，并提供了详细的执行记录和报告。其核心步骤包括：

1. **指令接收与分发 (PageAgent.ai)**: 根据指令类型调用相应的处理函数 (如 **aiAction**)。
2. **任务规划 (PageTaskExecutor 与 LLM/VLM)**:
 - **PageTaskExecutor** 获取当前页面上下文 (**WebUIContext**, 包含截图和DOM树)。
 - 通过 **@midscene/core** 的 **plan** 函数 (标准模式) 或 **@midscene/core/ai-model** 的 **vlmPlanning** 函数 (VLM模式) 与大语言模型交互，将用户指令和页面上下文发送给LLM/VLM。
 - LLM/VLM 返回一系列抽象的行动步骤 (**PlanningAction[]**) 和一个等效的YAML工作流 (**yamlFlow**)。这一步的核心在于 **@midscene/core** 内部的Prompt工程，它将页面信息、用户指令、历史上下文等结构化地呈现给AI。
3. **计划到执行的转换 (PageTaskExecutor.convertPlanToExecutable)**:
 - 将 **PlanningAction[]** 转换为具体的 **ExecutionTaskApply[]**。
 - 每个 **ExecutionTaskApply** 包含一个 **executor** 函数，该函数定义了如何通过 **this.page** 对象 (如 **PuppeteerWebPage** 或 **ChromeExtensionProxyPage**) 的方法 (如 **click**, **type**, **scroll**, **evaluateJavaScript**) 来实际操作页面。
 - Action 类型的任务会自动在执行前后进行截图。
4. **任务执行 (Executor)**:
 - **@midscene/core** 的 **Executor** 负责按顺序执行这些 **ExecutionTaskApply**，记录每个任务的输入、输出、日志、截图、耗时等信息。
5. **结果聚合与报告 (PageAgent.afterTaskRunning)**:
 - **PageAgent** 从 **Executor** 获取完整的执行记录 (**ExecutionDump**)。
 - 将这些记录聚合成 **GroupedActionDump**。
 - 序列化为JSON dump文件，并生成用户可读的HTML报告。

整个过程涉及了缓存机制 (**TaskCache**)、多模块协作 (**@midscene/core**, **@midscene/shared** 等) 以及与底层浏览器操作的紧密集成 (通过 **WebPage** 接口和 CDP 或 WebDriver协议)。